

README

README — n8n API Integration Workflow

Project Overview

This project is an automated workflow developed using [n8n](#) to generate a cryptocurrency market digest and send hourly notifications to Discord. It automates data collection, transformation, filtering, enrichment, and alert delivery using API integrations and conditional workflow logic. The goal is to monitor top crypto movements and notify only meaningful market changes.

APIs Used

➤ [CoinGecko API](#)

- Endpoints: /coins/markets, /coins/{id}
A free public API that provides real-time cryptocurrency market data including price, volume, and market capitalization. It is used for both initial data retrieval and detailed coin enrichment after selection. No authentication is required, making it suitable for automated workflows.

➤ [Discord Webhooks](#)

- A lightweight webhook API used to send real-time notifications to a Discord channel. It enables instant delivery of formatted crypto market summaries generated by the workflow.
-

Transformation Logic

- The workflow runs every hour using a Schedule Trigger. It first fetches live market data from CoinGecko, then processes the dataset using a Code node to extract and structure relevant fields. Coins are sorted based on market capitalization, and top results are identified.
 - A Limit node ensures only the highest-ranked coin is selected for further processing to reduce API calls and improve efficiency. A second API request retrieves detailed statistics for the selected coin.
 - An IF condition evaluates market movement using `price_change_percentage_24h > 0`. If TRUE, the system generates a formatted message and sends it to Discord. If FALSE, the workflow ends without sending notifications, ensuring only positive market signals are alerted.
 - The Code node normalizes API data by removing unnecessary fields and structuring only required attributes for consistent processing across nodes.
 - API responses are processed in a sequential flow to maintain data integrity, ensuring each step uses validated output from the previous node.
 - The workflow is optimized for efficiency by filtering and limiting data early in the pipeline, reducing unnecessary API calls and improving overall performance.
-

Error Handling

- The workflow implements robust error management using “Continue On Fail” to prevent execution stops during API or node failures. Additionally, a dedicated Error Trigger workflow captures runtime errors and sends structured error details (workflow name and issue description) to Discord. This ensures visibility of failures and prevents silent breakdowns in automation.
-

Technologies Used: [n8n](#), [CoinGecko API](#), [Discord Webhooks](#), JavaScript, REST APIs.